

# KubeML-Aware: Kubernetes-Native Rank-Aware Admission for Burst-Submitted ML Training

Amir Mirzaei Movahed and Majid Bafandeh

**Abstract**—Kubernetes makes mature placement decisions for Pods, but its default policy does not infer the relative value of machine-learning (ML) training jobs from their convergence and workload characteristics. This distinction matters when heterogeneous jobs arrive as a burst and compete on the same node. We present KubeML-Aware, a training-specific admission layer that leaves feasibility checks, node scoring, and binding to the default scheduler. The system collects a complete burst, derives a deterministic rank from six annotated characteristics, and controls scheduling eligibility through stable Pod scheduling gates. A fail-closed FastPath uses bounded CPU demand and fresh node telemetry to prefill the scheduler queue when progress can be established; a small tail optimizer may rearrange only the lowest-ranked jobs. We implemented the controller as an in-cluster service and evaluated the deployed path by executing real Kubernetes Jobs on a shared four-core K3s node. The controlled NumPy trainer performs matrix operations, gradient-buffer updates, synchronization copies, and checkpoint I/O. Five paired repetitions compared 12 heavy jobs under synchronized FIFO admission and KubeML-Aware. Mean job completion time fell from 31.047 to 28.755 s, a mean paired improvement of 7.31% (exploratory 95% CI [2.57%, 12.05%]). Makespan moved in the same direction (55.369 to 54.005 s, 2.47%) and decreased in all five pairs, but its paired interval includes zero, so we report it as directional rather than an established effect. These measurements support the feasibility of Kubernetes-native, rank-aware admission, but they are a small CPU-only pilot on a non-isolated node; they do not establish general performance for GPU or multi-node training.

**Index Terms**—Kubernetes, machine-learning training, scheduling gates, admission control, job ranking, job completion time.

## 1 INTRODUCTION

Training workloads rarely behave as interchangeable batch jobs. Even when two Pods request the same CPU and memory, their useful work can differ in duration, convergence rate, matrix or tensor scale, synchronization volume, and checkpoint frequency. Production traces show that this heterogeneity is a central difficulty in ML cluster management, alongside queueing delay and uneven accelerator utilization [1]. When a set of jobs is submitted together, the admission order can therefore change which jobs obtain early access to a contested node and how quickly the burst clears.

Kubernetes exposes a rich scheduling framework for deciding *where* a Pod may run. The default scheduler filters infeasible nodes, scores the remaining candidates, and

binds each Pod while respecting requests, taints, affinity, storage, and topology constraints [2]. It does not, however, derive a burst-relative priority from training behavior. A PriorityClass can express importance chosen by an operator, but it does not estimate that importance from ML features. Replacing the scheduler is one way to add such a policy, although it also creates the burden of reproducing or safely composing with mature placement logic.

This paper studies a narrower alternative: change when a training Pod becomes eligible for the existing scheduler, without replacing node selection. KubeML-Aware uses Pod Scheduling Readiness [3] to hold every member of an annotated burst behind a scheduling gate. Once the complete burst is present, a controller validates six features, calculates a deterministic order, and removes the gates according to a resource-aware policy. Gate removal does not bind a Pod or reserve a node; the default scheduler remains the placement authority.

The ranking model originates from the six-feature formulation in [4], but the deployed system differs materially from a manual-binding reproduction. It adds complete-burst validation, stable scheduling gates, resource-bounded manifests, fresh Metrics API feedback, crash-safe records, structured application-level start evidence, and a FastPath that avoids controller-induced gaps. The evaluation reports only measurements generated by this implementation. Published percentages from the source formulation are not treated as results of KubeML-Aware or as a head-to-head comparison because the two studies used different implementations, workloads, and environments.

The paper makes three contributions:

- a deterministic, burst-relative training rank with explicit validation, normalization, and tie handling;
- a Kubernetes-native controller that applies the rank through scheduling gates and a fail-closed, telemetry-aware FastPath while retaining `default-scheduler`; and
- a paired real-cluster pilot that executes identical physical CPU workloads in both arms and measures mean JCT and burst makespan from application-emitted events.

The evidence is deliberately bounded. The workload is a controlled training proxy rather than a neural-network benchmark; the cluster has one shared CPU node; and the five repetitions are insufficient for broad claims about

• Amir Mirzaei Movahed: [Amir.m.movahed@ut.ac.ir](mailto:Amir.m.movahed@ut.ac.ir).  
 • Majid Bafandeh: [majidb@aut.ac.ir](mailto:majidb@aut.ac.ir).

other workloads or environments. The aim is to establish an auditable production path and a credible first measurement, not to claim an optimal general-purpose ML scheduler.

## 2 BACKGROUND AND RELATED WORK

### 2.1 Kubernetes scheduling and admission

The contemporary Kubernetes scheduler is a plugin framework with separate scheduling and binding cycles. Extensions may influence queue order, feasibility, scoring, reservation, permission, and binding [2]. This architecture already supports sophisticated placement, but placement score and application utility are different quantities. A node score answers which feasible node is preferred; it does not state which training job should be admitted first to minimize a burst-level objective.

Scheduling gates provide a clean boundary for an external admission policy. A gated Pod stays outside the active scheduler queue until an authorized controller removes the gate [3]. The operation is one-way and changes eligibility only. Startup still depends on scheduler latency, image availability, Kubelet, the container runtime, and operating-system service. KubeML-Aware therefore records an application-emitted `EXECUTION_STARTED` event and does not equate gate order, binding order, or the Kubernetes `Running` phase with an exact kernel-level execution order.

The ecosystem is also moving toward workload-level scheduling. Kueue provides queueing, quota admission, priority, fair sharing, and integrations for batch and ML controllers [5]. Volcano and KAI add facilities such as gang scheduling, hierarchical queues, accelerator sharing, and topology-aware placement [6], [7]. Upstream Workload and PodGroup APIs are introducing related abstractions, although the current interfaces remain evolving features [8]. KubeML-Aware does not duplicate these systems. Its policy can be viewed as a compact ranking and gate-control component that could eventually sit behind a broader queue or quota manager.

### 2.2 ML-aware scheduling

ML schedulers have progressively incorporated signals that generic resource declarations omit. Tiresias uses attained service and runtime feedback to improve responsiveness without assuming exact job duration [9]; Gandiva exploits intra-job predictability for GPU time sharing and migration [10]. Sia jointly reasons about accelerator heterogeneity and goodput [11], while Shockwave plans under dynamically changing training throughput [12]. CASSINI focuses on communication patterns and network contention [13], and Rubick treats a job's distributed execution plan as a reconfigurable scheduling variable [14].

Recent Kubernetes-specific systems emphasize placement at larger scale. DRL-MLS learns placement decisions for ML training in a heterogeneous nine-node cluster [15]. Sakkara models rack and node topology to reduce interference and communication cost for distributed AI workloads [16]. A 2026 review similarly shows that current Kubernetes scheduling research spans placement, energy, interference, topology, and learning-based policies [17].

KubeML-Aware occupies a deliberately smaller design point. It neither reallocates accelerators nor learns a node-placement policy. It ranks an explicitly declared training burst, regulates scheduler eligibility, and preserves the default binding path. That restricted scope reduces integration risk, but also means that multi-node topology, GPU sharing, online adaptation, and tenant fairness remain outside the current contribution.

## 3 KUBEML-AWARE DESIGN

Fig. 1 provides the system context for readers unfamiliar with Kubernetes. A manifest is stored by the Kubernetes API; a controller may modify eligibility; the default scheduler selects a feasible node; and the node's Kubelet asks containerd to start the Pod. KubeML-Aware occupies only the eligibility step. It neither replaces the scheduler nor directly launches containers.

### 3.1 Control path and safety boundary

Fig. 2 summarizes the deployed path. A seeded generator creates a complete burst whose Pods share a run identifier and expected burst size. Each Pod carries six training annotations and the `ml.scheduler/release` scheduling gate. Resource requests and limits bound demand, and every Pod continues to name `default-scheduler`.

The controller accepts a burst only after exactly the declared number of Pods has appeared and membership has remained unchanged for a short quiet interval. Missing features, duplicate identifiers, inconsistent run metadata, non-finite values, or unexpected external gate changes fail the run rather than silently changing its meaning. Equal scores are resolved using creation time, UID, and name so that Kubernetes API list order cannot affect the result.

Every transition is persisted before the next mutation. The record contains Pod UIDs, feature values, ranks, intended release order, FastPath evidence, release times, and observed execution events. After a restart, the controller relists the burst and reconciles only operations whose UID and state still match. This record is also the basis for result collection.

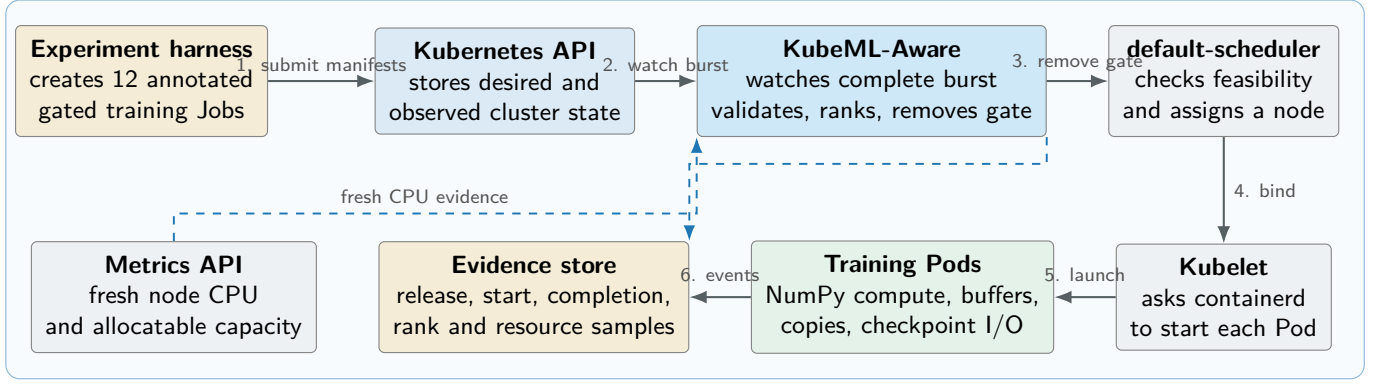
### 3.2 Burst-relative training rank

For job  $j$ , the feature vector is  $\mathcal{F} = \{T, R, M, G, C, P\}$ . In the implemented proxy,  $T$  is estimated training time,  $R$  controls loss reduction,  $M$  is the square-matrix dimension,  $G$  is the physical gradient-buffer size,  $C$  is the checkpoint interval, and  $P$  controls partition and synchronization work. Larger  $R$  and  $C$  are preferred; smaller  $T$ ,  $M$ ,  $G$ , and  $P$  are preferred. In particular,  $P$  is not a GPU count in the present CPU-only system.

Each feature is normalized within its own burst. For feature  $k$  of job  $j$ ,

$$\hat{x}_{k,j} = \begin{cases} \frac{x_{k,j} - x_k^{\min}}{x_k^{\max} - x_k^{\min}}, & k \in \{R, C\}, \\ \frac{x_k^{\max} - x_{k,j}}{x_k^{\max} - x_k^{\min}}, & k \in \{T, M, G, P\}. \end{cases} \quad (1)$$

If  $x_k^{\max} = x_k^{\min}$ , every job receives  $\hat{x}_{k,j} = 0.5$  for that feature. This makes the term neutral rather than rewarding an arbitrary job.



Kubernetes provides the API, placement engine, and node runtime.  
KubeML-Aware changes **when** a training Pod becomes eligible; it does not replace placement or container execution.

Fig. 1. Kubernetes context and the deployed KubeML-Aware control path. Solid arrows show the normal submission-to-execution path. Dashed arrows show telemetry and persistent experiment evidence. The shaded region is the single-node K3s cluster used in the pilot.

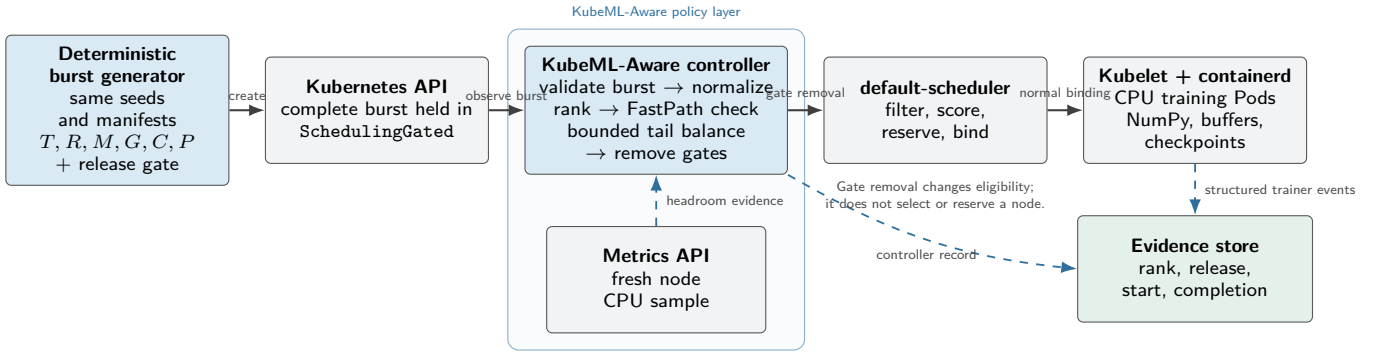


Fig. 2. Deployed KubeML-Aware architecture. The controller regulates eligibility, not placement. Gate removal hands a Pod to the normal Kubernetes scheduler; structured trainer events and controller records provide an auditable execution trace.

The base score is

$$S_j = 0.32\hat{T}_j + 0.28\hat{R}_j + 0.16\hat{M}_j + 0.12\hat{G}_j + 0.08\hat{C}_j + 0.04\hat{P}_j. \quad (2)$$

Jobs are sorted by decreasing  $S_j$ . The coefficients are fixed before evaluation and sum to 1. Scaling all coefficients by the same positive constant would not change the order. In the current generator,  $T$  is derived from the other five features, so the terms are not statistically independent. The score should consequently be read as an explicit policy, not a learned predictor or an optimality proof. Scores are also local to one burst and cannot be compared across runs.

### 3.3 Training FastPath

The conservative controller releases the highest-ranked Pod and waits for its application-level start marker before moving to the next job. This protects the intended prefix when startup conditions are uncertain, but it can leave the scheduler queue empty between releases. FastPath addresses that bubble without bypassing Kubernetes capacity checks.

A burst is FastPath-eligible only when it is training-only, uses no experimental pacing or reversed order, every application and initialization container has bounded CPU demand, and a fresh Metrics API sample is available [18].

Let  $u$  be measured node CPU utilization,  $D$  the sum of declared peak Pod demand,  $A$  allocatable CPU, and  $\tau = 0.85$  the configured threshold. The controller computes

$$u_{\text{projected}} = u + \frac{D}{A}. \quad (3)$$

If the whole burst fits, all gates are removed in ranked order. If at least one ranked Pod fits but the burst exceeds current headroom, the implementation still prefills the complete ranked queue; the default scheduler then keeps infeasible Pods pending according to their normal requests and constraints. If telemetry is stale or no bounded demand can be established, FastPath fails closed and the controller returns to the conservative path.

Before queue prefill, the controller may permute at most the four lowest-ranked jobs. It predicts a list schedule from  $T$ , keeps the high-priority prefix fixed, and chooses the suffix with the lowest predicted makespan, using cumulative completion time as a secondary criterion. At most  $4! = 24$  permutations are examined. Validation and feature normalization are  $O(N)$ , ranking is  $O(N \log N)$ , and the bounded suffix search does not change the asymptotic complexity.

TABLE 1  
Recorded Experimental Platform

| Component         | Recorded configuration                                       |
|-------------------|--------------------------------------------------------------|
| Virtual machine   | KVM; x86-64 architecture                                     |
| Processor         | 4 vCPU; Intel Xeon Gold 6138 at 2.00 GHz                     |
| Memory            | 7.57 GiB installed                                           |
| Operating system  | Ubuntu 22.04.5 LTS; Linux 5.15.0-177-generic                 |
| Orchestrator      | Single-node K3s/Kubernetes v1.36.2+k3s1; control-plane Ready |
| Container runtime | containerd 2.3.2-k3s2                                        |
| Training image    | registry.local/kubeml/ml-sim-job:0.3.0                       |
| Pod resources     | 1 vCPU request/limit; 128 MiB request; 1 GiB limit           |

## 4 WORKLOAD AND EXPERIMENTAL METHOD

### 4.1 Controlled training proxy

The experiment uses work-model version 2.0 and a containerized NumPy trainer [19]. For each step, the process multiplies partitions of two  $M \times M$  float matrices, updates a  $G$ -MiB gradient buffer, and performs  $P - 1$  physical buffer copies when  $P > 1$ . Synthetic loss follows

$$L_{s+1} = L_s e^{-R}, \quad (4)$$

and training stops at the convergence threshold or a bounded step budget. Every  $C$  steps, the process writes a bounded model-state payload and calls `fsync`.  $T$  is estimated from this versioned work model; it does not terminate the process.

The reported pilot uses the heavy profile only:  $R \in [0.03, 0.08]$ ,  $M \in [512, 1024]$ ,  $G \in [20, 50]$  MiB,  $C \in [20, 50]$  steps, and  $P \in [2, 4]$ . BLAS is restricted and attested to one thread. The workload exercises real CPU, memory, copying, and local I/O, but it is not a neural network, does not read a dataset, uses no GPU, and performs no inter-node communication. The shared parameters between the ranker and proxy make the pilot suitable for validating the control path, but not for establishing robustness to feature-estimation error.

### 4.2 Cluster and paired protocol

KubeML-Aware was implemented as a Python controller running inside Kubernetes, not as an offline simulator. All measurements reported here were produced by Kubernetes Jobs that reached completion on the deployed cluster and emitted structured execution-start and completion events. Table 1 records the platform observed during the experiment.

The operational node also hosted VPN, ingress, monitoring, and Knative-related services. Before each arm, the harness required two CPU samples at or below 35%, five seconds apart, and verified the protected VPN services. Their readiness and restart counts did not change during the experiment.

Five paired repetitions used seeds 8100–8104. Every repetition generated 12 source jobs, and the baseline and KubeML-Aware arms received identical feature values, trainer image, random seeds, resource declarations, and work-model version. The arms ran sequentially in separate namespaces and never overlapped. Arm order alternated:

TABLE 2  
Five-Pair Heavy-Training Pilot

| Metric      | FIFO     | KubeML-Aware | Paired change | 95% interval  |
|-------------|----------|--------------|---------------|---------------|
| Average JCT | 31.047 s | 28.755 s     | +7.31%        | [2.57, 12.05] |
| Makespan    | 55.369 s | 54.005 s     | +2.47%        | [−0.45, 5.40] |

the baseline ran first in repetitions 0, 2, and 4; KubeML-Aware ran first in repetitions 1 and 3.

Both arms used the same initial scheduling gate to remove sequential API-submission spread. The baseline removed gates in manifest/FIFO order after the full burst had been observed. The KubeML-Aware arm removed the same gates in its ranked, tail-balanced FastPath order. Both arms used `default-scheduler`; neither manually bound Pods. Thus the comparison isolates admission policy more closely than a baseline whose Pods become eligible as they are created.

For job  $j$ ,  $JCT_j = c_j - s_j$ , where  $s_j$  is Pod creation time and  $c_j$  is the application-level completion time. Makespan is  $\max_j(c_j) - \min_j(s_j)$ . The p95 JCT uses linear interpolation over the 12 job values. For metric  $m$  in repetition  $r$ , paired improvement is

$$I_{m,r} = 100 \frac{m_{\text{FIFO},r} - m_{\text{KubeML-Aware},r}}{m_{\text{FIFO},r}}. \quad (5)$$

Positive values mean that KubeML-Aware reduced the metric. We report the mean of the five within-pair percentages. Exploratory 95% intervals use a Student- $t$  interval over those five paired effects; given the small sample and shared host, they are descriptive rather than confirmatory.

## 5 RESULTS AND DISCUSSION

### 5.1 Paired pilot results

Table 2 and Fig. 3 summarize the five repetitions. Average JCT improved in every pair, with individual reductions from 3.32% to 12.79%. The mean paired improvement was 7.31%, and the exploratory 95% interval was [2.57%, 12.05%]. The corresponding grand means were 31.047 s for FIFO and 28.755 s for KubeML-Aware.

Makespan also improved in all five pairs, but by a smaller and more variable amount: 0.55% to 5.87%, with a mean paired improvement of 2.47%. Its exploratory interval crosses zero. The consistency of the observed direction is encouraging, but five runs on a shared node do not support a general throughput claim.

As a secondary diagnostic, p95 JCT changed from 52.896 to 53.270 s (a 0.75% paired regression). With only 12 jobs per arm, this percentile lies close to the maximum and is unstable; it is therefore not used as a headline outcome. The observation nevertheless indicates that a future policy should include an explicit tail-latency or aging objective.

Fig. 4 visualizes application-level execution intervals for repetition 0. The FIFO arm largely follows manifest order, whereas KubeML-Aware brings several jobs from later manifest positions into the first execution wave. This is direct evidence that the gate policy changed admission and observed start order. It is not, by itself, evidence that the six-feature equation, queue prefill, or tail optimizer caused a particular fraction of the final improvement.

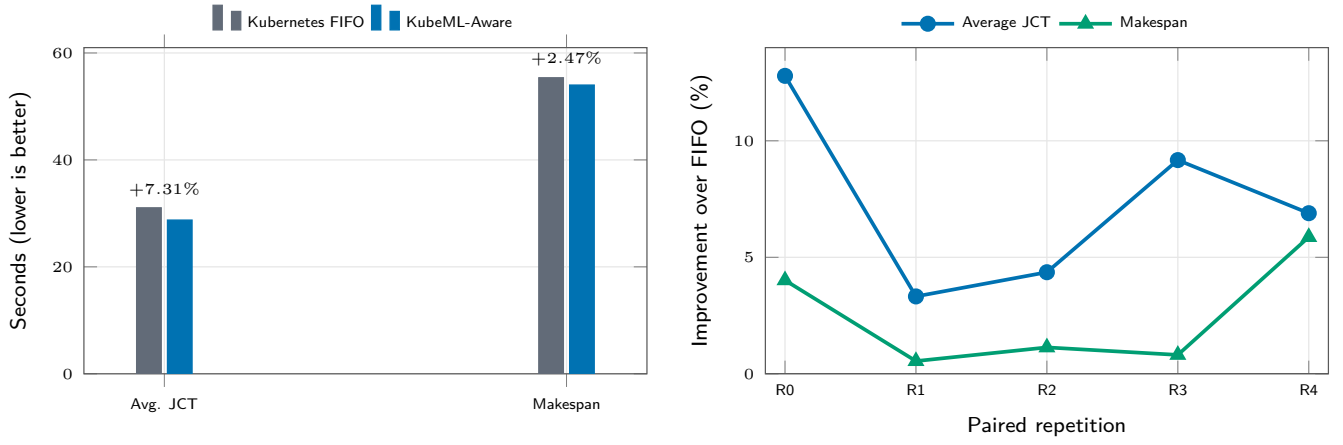


Fig. 3. Primary aggregate and paired pilot results. Bars show five-repetition means. Lines show within-pair improvement, where positive values indicate lower time under KubeML-Aware. Average JCT and makespan improved in all five pairs.

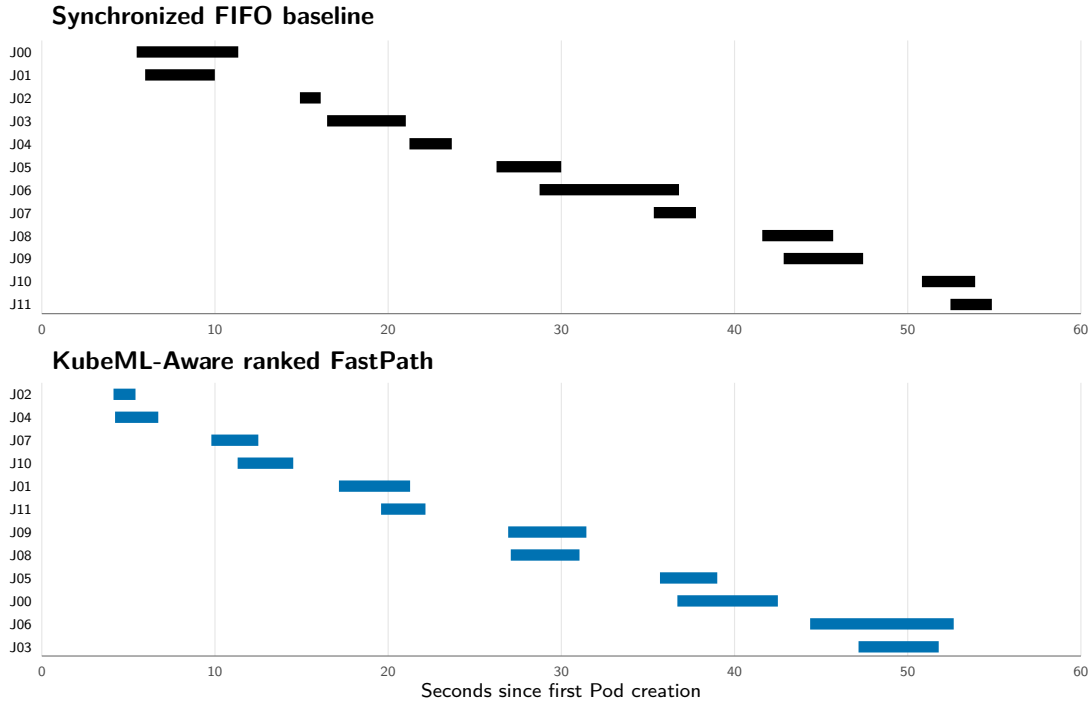


Fig. 4. Application-level execution intervals for repetition 0. Labels identify the same source-job indices in both arms; rows are ordered by observed execution start. The reordering is visible, while overlap shows that FastPath preserves concurrency rather than serializing the burst.

## 5.2 Threats to validity

The main threat is the environment. The evaluation node was an operational server, not an isolated benchmark host. Alternating arm order, paired inputs, cool-node checks, and unchanged protected-service state reduce systematic bias, but cannot remove background variation. We therefore label this run as a production-path pilot rather than a clean, isolated benchmark.

Second, the evaluation contains only five pairs and one 12-job heavy profile. The intervals in Table 2 depend on a small-sample normality assumption and should not be read as definitive significance tests. A tail percentile estimated from 12 observations per arm is especially weak.

Third, the controlled trainer shares its feature semantics with the ranking policy. That choice makes workload behavior auditable and prevents sleep-based simulation, but it may favor a rank whose inputs were designed around the same cost model. Real PyTorch or TensorFlow jobs can exhibit data stalls, framework overhead, GPU kernels, communication, and convergence error that this proxy does not represent.

Finally, the measured treatment combines six-feature ranking, queue prefill, and bounded tail balancing. No live duration-only, reversed-order, FastPath-without-tail, or alternative-weight arm was included. The pilot therefore measures the deployed policy as a whole and cannot assign

causality to one component. These limitations are part of the result boundary, not future details that can be assumed away.

## 6 CONCLUSION

KubeML-Aware shows that an ML-specific admission policy can be added to Kubernetes without replacing its placement engine. A controller ranks a complete annotated training burst, removes stable scheduling gates, and uses fresh resource evidence to avoid unnecessary admission gaps. On a shared four-core K3s node, the deployed policy reduced mean JCT by 7.31% across five paired 12-job runs (exploratory 95% CI [2.57%, 12.05%]); makespan moved in the same direction (2.47%) but its interval includes zero and is not an established effect. Both metrics decreased in every pair.

The result is promising but preliminary. The next evaluation should run on a dedicated cluster with more repetitions, predeclared ablations, and real framework workloads. GPU and multi-node experiments must give  $G$  and  $P$  measured communication and parallelism semantics; profiling should replace generator-provided estimates; and aging or an explicit tail objective should improve robustness. Source code and experiment artifacts are available at <https://github.com/AmirMirzaeiMovahed/KubeML-Aware>.

## REFERENCES

- [1] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding, "MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, 2022, pp. 945–960.
- [2] Kubernetes Authors, "Scheduling framework," Kubernetes Documentation, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>
- [3] —, "Pod scheduling readiness," Kubernetes Documentation, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-scheduling-readiness/>
- [4] M. Abdelshaheed and M. Ashour, "Intelligent scheduling of AI tasks in Kubernetes," in *Proceedings of the 35th International Conference on Computer Theory and Applications (ICCTA)*, 2025, pp. 24–31.
- [5] Kubernetes SIG Scheduling, "Kueue overview," Kueue Documentation, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://kueue.sigs.k8s.io/docs/overview/>
- [6] Volcano Community, "Unified scheduling," Volcano Documentation, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://volcano.sh/docs/keyfeatures/unifiedscheduling/>
- [7] NVIDIA, "KAI scheduler," GitHub repository, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://github.com/kai-scheduler/KAI-Scheduler>
- [8] Kubernetes Authors, "Workload API," Kubernetes Documentation, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/workload-api/>
- [9] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Guo, "Tiresias: A GPU cluster manager for distributed deep learning," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, 2019, pp. 485–500.
- [10] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang, F. Yang, and L. Zhou, "Gandiva: Introspective cluster scheduling for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 2018, pp. 595–610.
- [11] S. J. Subramanya, D. Arfeen, S. Lin, A. Qiao, Z. Jia, and G. R. Ganger, "Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 642–657.
- [12] P. Zheng, R. Pan, T. Khan, S. Venkataraman, and A. Akella, "Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 703–723.
- [13] S. Rajasekaran, M. Ghobadi, and A. Akella, "CASSINI: Network-aware job scheduling in machine learning clusters," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 2024, pp. 1403–1420.
- [14] X. Zhang, H. Zhao, W. Xiao, X. Jia, F. Xu, Y. Li, W. Lin, and F. Liu, "Rubick: Exploiting job reconfigurability for deep learning cluster scheduling," in *Proceedings of Machine Learning and Systems*, vol. 7, 2025.
- [15] S. Dong, B. Zheng, L. Pan, and S. Liu, "A reinforcement learning-based approach for scheduling ML training tasks in heterogeneous Kubernetes clusters," *Future Generation Computer Systems*, vol. 182, p. 108459, 2026.
- [16] J. Santos, A. Tantawi, P. Maniotis, C. Wang, O. Tardieu, T. Wauters, and F. D. Turck, "Sakkara: Intelligent topology-aware scheduling for Kubernetes in the age of AI," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 5437–5451, 2026.
- [17] M. Alhakimi and R. Latip, "Enhancing the Kubernetes scheduler: A state-of-the-art review from cloud to edge," *Computers*, vol. 15, no. 7, p. 458, 2026.
- [18] Kubernetes Authors, "Resource metrics pipeline," Kubernetes Documentation, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/>
- [19] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [20] A. M. Movahed, "KubeML-Aware: Source code and experiment artifacts," GitHub repository, 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://github.com/AmirMirzaeiMovahed/KubeML-Aware>